

架构候选方案对比表

团队：第67组 日期：2026/5/2

1. 候选方案

1.1 候选方案A

前后端分离 + 分层单体架构（MVC）+ 进程内轻量事件驱动



架构方案分析：

优势：

- **SEO 友好：**搜索引擎能直接抓取完整的 HTML 页面内容，对于校园二手交易、需求发布等场景更有优势，商品信息更容易被检索到
- **首屏加载快：**不需要先加载空 HTML 再异步请求数据渲染，页面直达，对移动端弱网络环境更友好，提升用户体验
- **开发调试简单：**没有前后端接口联调问题，不用维护 API 文档，路由跳转直接由后端控制，减少沟通成本
- **安全性更高：**敏感数据直接在模板中渲染，不暴露额外的 API 接口，减少了攻击面，降低数据泄露风险

- **适合校园场景：**校园网环境复杂，减少前端请求次数（1 次请求拿到完整页面 vs 多次请求），降低网络不稳定带来的影响

劣势：

- 前后端耦合度高，修改前端需要改动后端模板文件
- 页面跳转需整体刷新，用户体验不如 SPA 流畅
- 难以支持多端复用（如需开发小程序需重写前端）
- 未来拆分为微服务时，View 层需要完全重构成 API 模式

1.2 候选方案B

前后端分离的分层单体架构



架构方案分析：

优势：

- **单体架构为主，降低开发与部署复杂度：**整体为单一应用，一次开发、一次部署，无需处理分布式服务拆分、注册发现等复杂问题，适配学生团队能力
- **严格分层设计，提升可维护性：**各层职责边界清晰，修改某一层逻辑（如前端框架更换、数据库优化）不会影响其他层，便于问题定位与后期迭代
- **轻量异步事件，优化核心体验：**仅在站内通知、信用分更新等非核心流程，采用进程内异步事件机制，不引入外部消息队列，既避免阻塞核心接口，又不增加架构复杂度
- **适配项目性能要求：**通过 Redis 缓存热点数据、数据库索引优化，可轻松支撑 500 人同时在线，确保核心操作响应时间 < 2 秒，满足校园场景使用需求
- **隐私与安全易落地：**匿名发布、匿名评价、敏感词过滤等需求，可在业务逻辑层统一实现，分层结构便于权限管控与安全校验
- **可演进性强：**采用模块化设计，未来若用户量提升、业务扩展，可基于现有模块平滑拆分为微服务，无需推倒重构
- **多端复用能力：**基于 uni-app 开发，一套代码可编译为 H5、微信小程序、App，满足校园用户多端访问需求

劣势：

- 需要维护前后端接口文档，联调工作量较大
- 团队成员需同时掌握 Vue.js 和 Spring Boot 两套技术栈
- 首屏加载需要两次请求（HTML 壳 + 数据接口），弱网环境下体验略差
- SEO 不如服务端渲染方案友好（可后期通过 SSR 或预渲染优化）

2. 对比结果

对比维度	方案 A	方案 B
开发复杂度	较低 无前后端联调，一套 Python 技术栈，开发效率高，改动即生效	中等 需要维护 API 文档、前后端联调、前端独立构建，但职责分离清晰便于分工
可维护性	中等 前后端代码耦合在同一项目中，改前端需要动后端模板，多人协作易冲突	较高 前后端独立部署，代码仓库分离，修改互不影响，分层清晰便于定位问题
可扩展性	中等 未来拆微服务时，View 层需要完全重构成 API 模式，迁移成本高	较高 天然就是 API 模式，未来可按模块平滑拆分为微服务，无需推倒重构
性能	较高 首屏一次请求返回完整 HTML，后端渲染压力稍大，但请求次数少	较高 首屏可能多一次请求（HTML 壳 + 数据接口），但后续交互无刷新，带宽消耗更小
团队学习成本	很低 全栈 Python + Jinja2 + SQLAlchemy，一套技术栈，适合学生团队快速上手	中等 需要同时掌握 Vue.js（前端）和 Spring Boot（后端），知识面更宽，学习周期更长
10 周内可行性	很高 开发速度快，无前端构建工具链，改动即生效，适合快速迭代和敏捷开发	较高 需要前后端并行开发，联调可能占用时间，但分层清晰便于分工，可同步推进

3. 综合评估与建议

3.1 方案适用场景分析

场景特征	推荐方案
团队前端经验不足	方案 A
需要开发多端（小程序 + H5 + App）	方案 B
偏内容型页面（公告、商品展示）	方案 A
偏工具型交互，注重体验	方案 B
未来需要扩展为微服务	方案 B

场景特征	推荐方案
时间限制在10周内	方案A

3.2 最终选择

方案 B

推荐理由：

- 技术适用性：Spring Boot + Vue.js 是目前企业级开发的主流技术栈
- 多端复用能力：uni-app 可一套代码编译到 H5 和小程序，满足校园用户微信内访问的实际需求
- 可演进性强：即使本学期只完成核心功能，未来扩展（如拆微服务、加新功能）不需要推翻重来
- 利于分工：有前端基础的同学负责 Vue，有后端基础的同学负责 Spring Boot，可并行开发提高效率